



TECHNISCHE UNIVERSITÄT
BERGAKADEMIE FREIBERG

Die Ressourcenuniversität. Seit 1765.

PERSONAL PROGRAMMING PROJECT

Winter Semester 2020/21

PARAMETER IDENTIFICATION OF ELASTIC- VISCOPLASTIC / ELASTIC-PLASTIC MATERIAL AND ITS FEM IMPLEMENTATIONS

By

Manikandan Valliappan

Exam no : 63965

Email : manikandan.valliappan@student.tu-freiberg.de

April 15, 2021

TABLE OF CONTENTS

1. INTRODUCTION	3
2. THEORITICAL BACKGROUND OF THE IMPLEMENTATION	3
2.1. PARAMETER IDENTIFICATION	3
2.1.1. NON-LINEAR LEAST SQUARE PROBLEMS	4
2.1.2. GAUSS NEWTON METHOD.....	5
2.1.3. LEVENBERG – MARQUARDT METHOD.....	6
3. FINITE ELEMENT FORMULATIONS	8
3.1. ELASTIC VISCOPLASTIC MATERIAL	9
3.2. ELASTIC PLASTIC MATERIAL	13
4. PROGRAM STRUCTURE	16
4.1. PARAMETER IDENTIFICATION FLOWCHART	16
4.2. FEM FLOWCHART	17
5. RESULTS OF TESTS AND DISCUSSION.....	18
6. TASKS PROPOSED Vs TASKS ACHIEVED	34
7. USER MANUAL.....	36
8. LITERATURE	39
9. GIT LOG	39

1. INTRODUCTION

Constitutive models for Elastic-Viscoplastic/Elastic-Plastic materials require many parameters to determine the behaviour of the material with accuracy. Commonly, material parameters are determined from laboratory experiments. However, determining the material parameters for the simulation process is quite problematic. This work is a step towards optimization of material parameters using Gauss Newton and Levenberg Marquart algorithms.

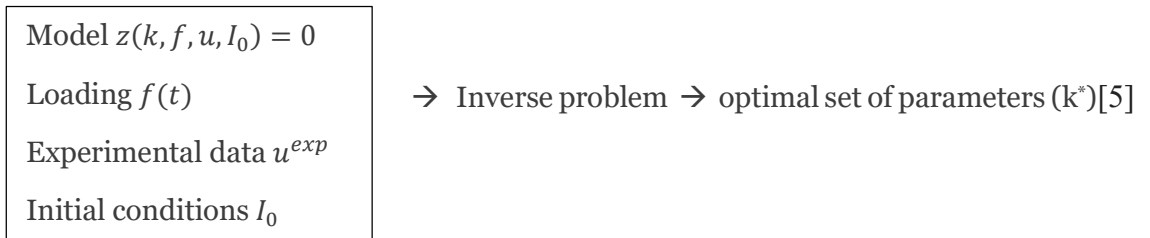
This work also includes the finite element implementation of Elastic-Viscoplastic/Elastic-Plastic material. Compression of the constrained cylinder is taken as the numerical problem and implemented into a finite element algorithm. Furthermore, material parameters are optimized and compared by using the Gauss Newton and Levenberg-Marquardt method.

This work aims to optimize the unknown material parameters and could be referred to as “optimum estimates” to the FEM model. Optimizing the complex material parameters by the mathematical approach motivated me to choose this topic.

2. THEORITICAL BACKGROUND OF THE IMPLEMENTATION

2.1. PARAMETER IDENTIFICATION

The term parameter identification is coined to identify the optimal parameters of the material model based on experimental data. Commonly material parameters are measured independently from specific experiments. For complex problems like nonlinear inelastic constitutive models is quite problematic. In such cases, the parameter identification method plays an important role. In this work, the inverse problem is taken into consideration to find the optimal set of material parameters [5].



In indirect or inverse problems, loading history, initial conditions by comparison of simulated and measured material response are given as input to determine the optimal set of parameters. Since the set of material data is larger than the set of parameters to be found. Therefore the inverse problem is overdetermined, a unique solution does not exist. Hence optimal fit between experimental and simulated data, in the sense of least-square minimization.

2.1.1. NON-LINEAR LEAST SQUARE PROBLEMS

Non-linear least squares are the form of least squares analysis used for optimal fit of material data (m) with non-linear the set of unknown parameters (n). (i.e. $m \geq n$)

Specific form of Non-linear least squares problem is,

$$f(k) = \frac{1}{2} \sum_{i=1}^m r_i^2(k) \quad (2.1)$$

Where $r_i(k)$ is the smooth function from R^n to R . Each r_i is a residual and we assume that $m \geq n$.

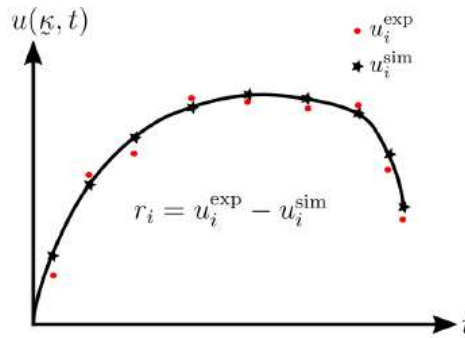


Figure 1: Two discretized curves and the corresponding residuals [5].

The individual components of r_i can be assembled as a vector of residuals,

$$r(k) = [r_1(k), r_2(k), \dots, r_m(k)]^T \quad (2.2)$$

$$\text{Objective function : } f(k) = \frac{1}{2} \|r(k)\|^2 \quad (2.3)$$

The derivatives of $f(k)$ can be expressed as the Jacobian of r (i.e. $m \times n$ matrix of first partial derivatives)

$$J(k) = \left[\frac{\partial r_i}{\partial k_j} \right] = \begin{bmatrix} \frac{\partial r_1}{\partial k_1} & \dots & \frac{\partial r_1}{\partial k_n} \\ \dots & \dots & \dots \\ \frac{\partial r_m}{\partial k_1} & \dots & \frac{\partial r_m}{\partial k_n} \end{bmatrix}$$

Where $i = 1, \dots, m$ data points and $j = 1, \dots, n$ parameters.

Gradient of $f(k)$:

$$\nabla f(k) = \sum_{i=1}^m r_i(k) \nabla r_i(k) = J(k)^T r(k) \quad (2.4)$$

Hessian of $f(k)$:

$$\begin{aligned}\nabla^2 f(k) &= \sum_{i=1}^m r_i(k) \nabla r_i(k)^T + \sum_{i=1}^m r_i(k) \nabla^2 r_i(k)^T \\ \nabla^2 f(k) &= J(k)^T J(k) + \sum_{i=1}^m r_i(k) \nabla^2 r_i(k)^T\end{aligned}\quad (2.5)$$

The major algorithms for minimizing the specific form of nonlinear least square problem fit are line search and trust region methods. They are based on Newton or Quasi-Newton approaches.

2.1.2. GAUSS NEWTON METHOD

Gauss Newton is modification of Newton method with line search. Excluding the second order term from standard Newton equations to find a search direction s_i

By solving $J_i^T J_i s_i^{GN} = -J_i^T r_i$, we can find the search direction. This can be solved using `s = numpy.linalg.lstsq(J.T,-r)` in python.

Gauss - Newton algorithm[5] :

```
Initialize  $k_0 = [k_1 \dots k_n]$ ,  $f_{tot}$ ,  $i_{max}$ ,  $i = 1$ 
set  $r_0 = \text{residual}(k_0)$ 
set  $f_0 = \frac{1}{2} \|r_0\|^2$ 
set  $j_0 = \text{jacobian}(k_0)$ 
set  $m_0 = f_0$ 
while  $\Delta f > f_{tot}$  and  $i < i_{max}$  :
    set  $\Delta k = \text{lstsq}([j_{i-1}]^T, -r_{i-1})$ 
    set  $k_i = k_{i-1} + \Delta k$ 
    set  $r_i = \text{residual}(k_i)$ 
    set  $f_i = \frac{1}{2} \|r_i\|^2$ 
    set  $j_i = \text{jacobian}(k_i)$ 
    set  $\Delta f = |f_i - f_{i-1}|$ 
    i = i+1
```

2.1.3. LEVENBERG – MARQUARDT METHOD

Levenberg Marquardt's method works on a trust-region framework. Hence the following problem is to solve for each iteration for the spherical trust-region radius Δ_i .

$$\min_s = \frac{1}{2} \|J_i s + r_i\|^2 \quad \text{with } \|s\| \leq \Delta_i \quad (2.6)$$

The next step is to find a trust-region radius Δ_i . Agreement between model function m_i and the objective function from previous iterations is,

$$\rho_i = \frac{f(k_i) - f(k_i + s_i)}{m_i(0) - m_i(s_i)} \quad (2.7)$$

- Where denominator represents predicted reduction and nominator represents actual reduction. If ρ_i is negative, the new objective value is greater than the current value and the step must be rejected.
- If ρ_i is closed to 1, there is a good agreement between the model m_i and the objective function f , so the trust-region can be increased.
- If ρ_i is closed to zero or even negative, the trust-region must be reduced.

Model function :

$$m_i(s) = \|r_i\|^2 + s^T J_i r_i + \frac{1}{2} s^T J_i^T J_i s \quad (2.8)$$

If the search direction s^{GN} from gauss Newton method lies inside the trust region then this s^{GN} also solves the trust region problem or $\lambda > 0$ such that $s = s^{LM}$ of trust region problem satisfies the constraint $\|s\| = \Delta$ and

$$(J^T J + \lambda I)s = -J^T r \quad (2.9)$$

By increasing the λ the term $(J^T J + \lambda I)$ can be positive definite, which decreases the accuracy of the model and steps with a certain length are more likely to be rejected.

Levenberg – Marquardt algorithm[5] :

Initialize $k_0 = [k_1 \dots \dots k_n]$, $\lambda = 0.001$, f_{tot} , $\eta = 0.25$, i_{max} , j_{max} , $i = 1$, $j = 1$
 set r_0 = residual (k_0)
 set $f_0 = \frac{1}{2} \|r_0\|^2$
 set j_0 = jacobian (k_0)

set $m_0 = f_0$

while $\Delta f > f_{tot}$ and $i < i_{max}$ and $j < j_{max}$:

set $\Delta k = \text{lstsq}([j_{i-1}\sqrt{\lambda} \ I]^T, -[r_{i-1} \ 0]^T)$

set $m_i = f_{i-1} + \Delta k^T j_{i-1} + \frac{1}{2} \Delta k^T j_{i-1}^T j_{i-1} \Delta k$

set $k_i = k_{i-1} + \Delta k$

set $r_i = \text{residual}(k_i)$

set $f_i = \frac{1}{2} \|r_i\|^2$

set $j_i = \text{jacobian}(k_i)$

set $\Delta f = |f_i - f_{i-1}|$

set $\rho_i = (f_i - f_{i-1}) / (m_{i-1} - m_i)$

if $\rho_i \leq \frac{1}{4}$:

set $\lambda_i = 10\lambda_{i-1}$

else:

if $\rho_i \geq \frac{3}{4}$:

set $\lambda_i = 0.1\lambda_{i-1}$

else:

set $\lambda_i = \lambda_{i-1}$

if $\rho_i > \eta$:

accept step

Set $j = 1$

Set $i = i+1$

else:

reject step

set $k_i = k_{i-1}$

set $j_i = j_{i-1}$

end while

3. FINITE ELEMENT FORMULATIONS

The compression of a cylinder between two plates have friction causes the ends to be constrained from motion in the radial direction. This constraining produces stress concentration at the outer edges of the cylinder. These concentrations imposed on the nominal stresses create stress variations throughout the cylinder. Since there are variations in more than one direction, predicting the resulting behaviour requires multidimensional analysis.

This problem is chosen for symmetry in geometry and boundary conditions of the cylinder allow for simplification from 3D analysis to 2D axisymmetric analysis[1].

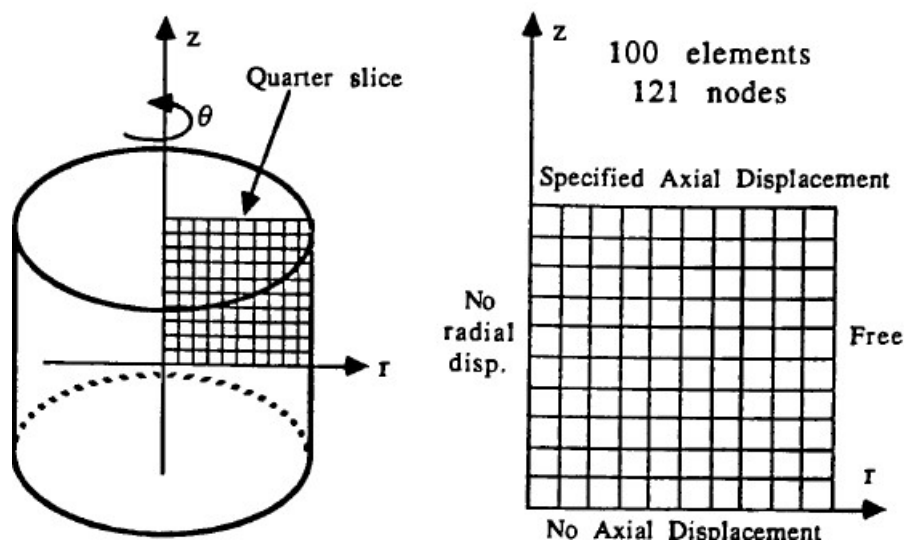


Figure 2: Geometry and Boundary conditions of 2D axisymmetric cylinder [1]

In this project, 4 elements with 9 nodes are implemented.

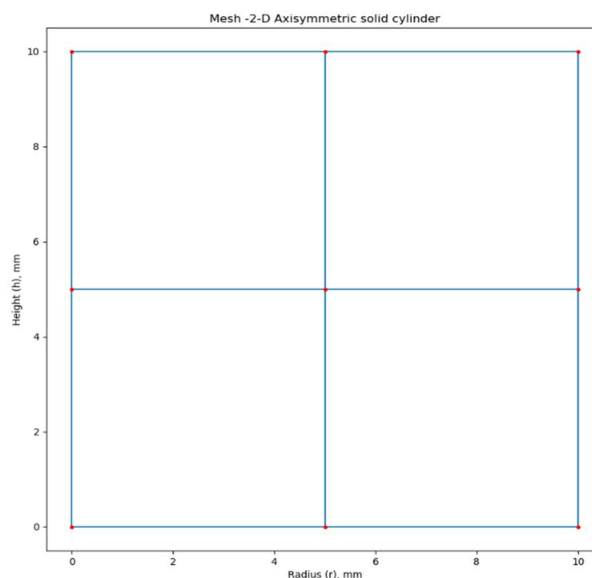


Figure 3 : Representation of mesh : Four elements with nine nodes

3.1. ELASTIC VISCOPLASTIC MATERIAL

Weak form,

Principle of the rate of virtual work applied to an element and assuming isothermal quasi static loading with negligible body forces,

$$\int [\delta \mathbf{E}]^T [\dot{\boldsymbol{\sigma}}] .d\Omega = \int [\delta \mathbf{U}]^T [\dot{\mathbf{F}}] .dS$$

Where $[\delta \mathbf{E}]^T$ is the transpose of strain variation, $\dot{\boldsymbol{\sigma}}$ is the stress rate, Ω is the domain of the element, $[\delta \mathbf{U}]^T$ is the transpose of displacement variation, $\dot{\mathbf{F}}$ is the external force rate and S is the surface.

For axisymmetric case, strain, and stress rate (Voigt notation) are defined as

$$\begin{bmatrix} \varepsilon_r \\ \varepsilon_z \\ \varepsilon_\theta \\ \gamma_{rz} \end{bmatrix} = \begin{bmatrix} \frac{\partial u}{\partial r} \\ \frac{\partial v}{\partial z} \\ \frac{u}{r} \\ \frac{\partial v}{\partial z} + \frac{\partial u}{\partial r} \end{bmatrix} = \begin{bmatrix} \frac{\partial N}{\partial r} & 0 \\ 0 & \frac{\partial N}{\partial z} \\ \frac{N}{r} & 0 \\ \frac{\partial N}{\partial z} & \frac{\partial N}{\partial r} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = [\mathbf{B}][\mathbf{U}]$$

$$\dot{\boldsymbol{\sigma}} = \begin{bmatrix} \dot{\sigma}_r \\ \dot{\sigma}_z \\ \dot{\sigma}_\theta \\ \dot{\tau}_{rz} \end{bmatrix}$$

Where $\mathbf{B}$ is the stress-strain relationship matrix. $\mathbf{u}$ and $\mathbf{v}$ are displacements in radial and axial direction. The four stress components are radial, axial, hoop, and shear.

Nodal displacement vector,

$$[\varphi] = \begin{bmatrix} N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 & 0 \\ 0 & N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 \end{bmatrix}$$

For the isoparametric bilinear quadrilateral, four Shape functions are,

$$N_1 = \frac{1}{4} * (1 - \xi) * (1 - \eta)$$

$$N_2 = \frac{1}{4} * (1 + \xi) * (1 - \eta)$$

$$N_3 = \frac{1}{4} * (1 + \xi) * (1 + \eta)$$

$$N_4 = \frac{1}{4} * (1 - \xi) * (1 + \eta)$$

The coordinates ξ, η are called natural coordinates. These shape functions map the element from physical domain to the natural coordinates of the parametric domain.

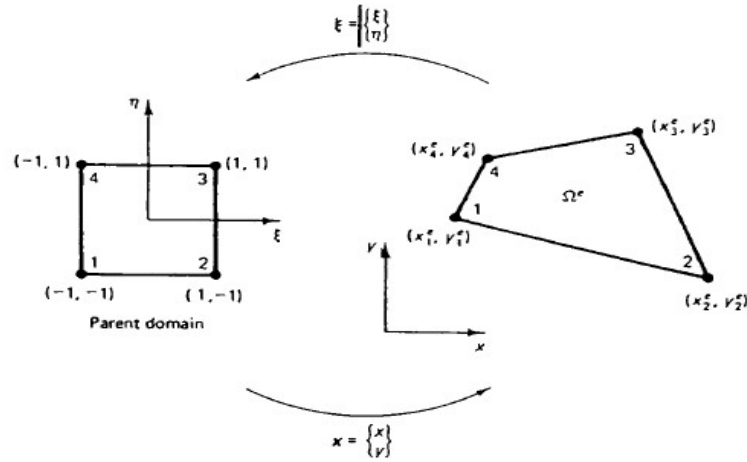


Figure 4: Isoparametric mapping and shape functions for bilinear quadrilateral element [1]

Computation of Material state variables (J2 von-mises elastic-viscoplasticity)

Von mises Viscoplastic model with nonlinear hardening Algorithm[4] :

$$dev(\epsilon_{n+1}) = \epsilon_{n+1} - \frac{1}{3}tr(\epsilon_{n+1}).I$$

$$dev(\sigma_{n+1}^{trial}) = 2\mu.dev(\epsilon_{n+1}^{trial}) = 2\mu. [dev(\epsilon_{n+1}) - \epsilon_n^{vp}]$$

$$\beta_{n+1}^{trial} = h.\alpha_n + (\Delta y.(1 - e^{-\eta\alpha}))$$

$$\xi_{n+1}^{trial} = dev(\sigma_{n+1})$$

$$n_{n+1}^{trial} = \frac{\xi_{n+1}^{trial}}{\|\xi_{n+1}^{trial}\|}$$

$$\phi_{n+1}^{trial} = \|\xi_{n+1}^{trial}\| - \sqrt{\frac{2}{3}} \cdot [\sigma_{y0} + \beta_{n+1}^{trial}]$$

Check for elastic- Viscoplastic loading

if $\phi_{n+1}^{trial} < 0$:

(elastic)

$$dev(\sigma_{n+1}) = dev(\sigma_{n+1}^{trial})$$

$$C_{n+1}^{dev} = 2\mu\mathbb{P}_{sym}$$

$$\epsilon_{n+1}^{vp} = \epsilon_n^{vp}$$

$$\alpha_{n+1}^{vp} = \alpha_n^{vp}$$

else:

(radial return mapping (Viscoplastic correction))

$$\gamma_{n+1} = \frac{\phi_{n+1}^{trial}}{2\mu \frac{\tau}{\Delta t} + 2\mu + \frac{2}{3}h} \quad \text{with } \tau = \frac{\eta}{2\nu}$$

$$dev(\sigma_{n+1}) = dev(\sigma_{n+1}^{trial}) - 2\mu \cdot \gamma_{n+1} \cdot n_{n+1}^{trial}$$

$$\varepsilon_{n+1}^{vp} = \varepsilon_n^{vp} + 2\mu \cdot \gamma_{n+1} \cdot n_{n+1}^{trial}$$

$$\alpha_{n+1}^{vp} = \alpha_n^{vp} + \sqrt{\frac{2}{3}} \cdot \gamma_{n+1}$$

$$\beta_1 = 1 - \frac{\phi_{n+1}^{trial}}{\|\xi_{n+1}^{trial}\|} \cdot \frac{1}{\frac{\tau}{\Delta t} + 1 + \frac{h}{3\mu}}, \quad \beta_2 = \left[1 - \frac{\phi_{n+1}^{trial}}{\|\xi_{n+1}^{trial}\|} \right] \cdot \frac{1}{\frac{\tau}{\Delta t} + 1 + \frac{h}{3\mu}}$$

$$C_{n+1}^{dev} = 2\mu\beta_1 \mathbb{P}_{sym} - 2\mu\beta_2 n_{n+1}^{trial} \otimes n_{n+1}^{trial}$$

Add spherical contributions to stress and moduli

$$(\sigma_{n+1}) = dev(\sigma_{n+1}) + k \, tr(\varepsilon_{n+1}^{total}) \cdot I$$

$$C_{n+1} = C_{n+1}^{dev} + k \, I \otimes I$$

Gauss quadrature

Integration formula for an element,

$$\int f(x, y) d\Omega = \int_{-1}^1 \int_{-1}^1 f(x(\xi, \eta), y(\xi, \eta)) J(\xi, \eta) \cdot d\xi d\eta$$

For numerical integration, gauss quadrature method is used.

$$\int f(x, y) d\Omega = \sum_{i=1}^M f(x(\xi_i, \eta_i), y(\xi_i, \eta_i)) J(\xi_i, \eta_i) * Weight$$

Where f is a function, and J is the jacobian determinant.

Two sampling points in each unit coordinate direction hence locations and weights of gauss points are,

α	1	2	3	4
$\xi(\alpha)$	$-\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$-\frac{1}{\sqrt{3}}$
$\eta(\alpha)$	$-\frac{1}{\sqrt{3}}$	$-\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$
Weight	1	1	1	1

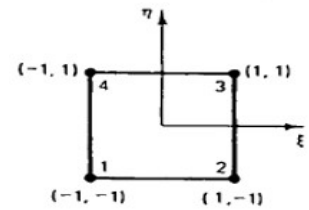


Figure 5: Integration points [1]

Table 1: Locations and weight of gauss points

Applying the gauss quadrature method of numerical integration to find k^{ele} and $F_{internal}^{ele}$

$$k^{ele} = weight * B^T * C_t * B * 2 * \pi * (N_1 r_1 + N_2 r_2 + N_3 r_3 + N_4 r_4) * || \text{Jacobian} ||$$

If (*elastic*) :

$$F_{internal}^{ele} = weight * B^T * \sigma * 2 * \pi * (N_1 r_1 + N_2 r_2 + N_3 r_3 + N_4 r_4) * || \text{Jacobian} ||$$

else (*viscoplastic*):

$$F_{internal}^{ele} = weight * B^T * \Delta \sigma * 2 * \pi * (N_1 r_1 + N_2 r_2 + N_3 r_3 + N_4 r_4) * || \text{Jacobian} ||$$

Connectivity matrix is multiplied with elemental stiffness and internal forces to find the global stiffness matrix and internal forces.

Since it is a displacement driven problem, $F^{external} = 0$

After finding global stiffness matrix and internal forces , we apply the essential boundary conditions.

1. $U_z = 0$ at $z = 0$
2. $U_r = 0$ at $r = 0$
3. $U_z = \text{Compresssion load at } z = \text{maximum height}$

Reduced global stiffness matrix and internal forces,

$$G = -F_{internal}^{global}, \quad F_{external}^{global} = 0$$

Now we solve for displacement,

$$\Delta U = K^{-1}(G)$$

If (elastic) [3]:

$$U_{k+1}^m = U_k^m + \Delta U$$

Else(viscoplastic) [3]:

$$dU = U_k^m + \Delta U$$

$$U_{k+1}^m = U_k^m + dU$$

Where k is Newton Raphson iteration and m is timestep

This process continues until it is Converged with Newton Raphson method.

$$||G|| < 0.005 ||F_{internal}^{global}||, \quad ||\Delta U|| < 0.005 ||U||$$

3.2. ELASTIC PLASTIC MATERIAL

Weak form,

Principle of the rate of virtual work applied to an element and assuming isothermal quasi static loading with negligible body forces,

$$\int [\delta \mathbf{E}]^T [\boldsymbol{\sigma}] .d\Omega = \int [\delta \mathbf{U}]^T [\mathbf{F}] .dS$$

Where $[\delta \mathbf{E}]^T$ is the transpose of strain variation, $\boldsymbol{\sigma}$ is the stress, Ω is the domain of the element, $[\delta \mathbf{U}]^T$ is the transpose of displacement variation, $\mathbf{F}$ is the external force and S is the surface.

For axisymmetric case, strain, and stress rate (Voigt notation) are defined as

$$\begin{bmatrix} \varepsilon_r \\ \varepsilon_z \\ \varepsilon_\theta \\ \gamma_{rz} \end{bmatrix} = \begin{bmatrix} \frac{\partial u}{\partial r} \\ \frac{\partial v}{\partial z} \\ \frac{u}{r} \\ \frac{\partial v}{\partial z} + \frac{\partial u}{\partial r} \end{bmatrix} = \begin{bmatrix} \frac{\partial N}{\partial r} & 0 \\ 0 & \frac{\partial N}{\partial z} \\ \frac{N}{r} & 0 \\ \frac{\partial N}{\partial z} & \frac{\partial N}{\partial r} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = [\mathbf{B}][\mathbf{U}]$$

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_r \\ \sigma_z \\ \sigma_\theta \\ \tau_{rz} \end{bmatrix}$$

Where $\mathbf{B}$ is the stress-strain relationship matrix. $\mathbf{u}$ and $\mathbf{v}$ are displacements in radial and axial direction. The four stress components are radial, axial, hoop, and shear.

Nodal displacement vector,

$$[\boldsymbol{\varphi}] = \begin{bmatrix} N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 & 0 \\ 0 & N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 \end{bmatrix}$$

For the isoparametric bilinear quadrilateral, four Shape functions are

$$N_1 = \frac{1}{4} * (1 - \xi) * (1 - \eta)$$

$$N_2 = \frac{1}{4} * (1 + \xi) * (1 - \eta)$$

$$N_3 = \frac{1}{4} * (1 + \xi) * (1 + \eta)$$

$$N_4 = \frac{1}{4} * (1 - \xi) * (1 + \eta)$$

The coordinates ξ, η are called natural coordinates. These shape functions map the element from physical domain to the natural coordinates of the parametric domain.

Von mises Plastic model with nonlinear hardening Algorithm[4] :

$$dev(\epsilon_{n+1}) = \epsilon_{n+1} - \frac{1}{3} tr(\epsilon_{n+1}) \cdot I$$

$$dev(\sigma_{n+1}^{trial}) = 2\mu \cdot dev(\epsilon_{n+1}^{trial}) = 2\mu \cdot [dev(\epsilon_{n+1}) - \epsilon_n^p]$$

$$\beta_{n+1}^{trial} = h \cdot \alpha_n + (\Delta y \cdot (1 - e^{-\eta\alpha}))$$

$$\xi_{n+1}^{trial} = dev(\sigma_{n+1})$$

$$n_{n+1}^{trial} = \frac{\xi_{n+1}^{trial}}{\|\xi_{n+1}^{trial}\|}$$

$$\phi_{n+1}^{trial} = \|\xi_{n+1}^{trial}\| - \sqrt{\frac{2}{3}} \cdot [\sigma_{y0} + \beta_{n+1}^{trial}]$$

Check for elastic- plastic loading

if $\phi_{n+1}^{trial} < 0$:

(elastic)

$$dev(\sigma_{n+1}) = dev(\sigma_{n+1}^{trial})$$

$$C_{n+1}^{dev} = 2\mu \mathbb{P}_{sym}$$

$$\epsilon_{n+1}^p = \epsilon_n^p$$

$$\alpha_{n+1}^p = \alpha_n^p$$

else:

(radial return mapping (plastic correction))

$$\gamma_{n+1} = \frac{\phi_{n+1}^{trial}}{2\mu + \frac{2}{3}h}$$

$$dev(\sigma_{n+1}) = dev(\sigma_{n+1}^{trial}) - 2\mu \cdot \gamma_{n+1} \cdot n_{n+1}^{trial}$$

$$\epsilon_{n+1}^p = \epsilon_n^p + 2\mu \cdot \gamma_{n+1} \cdot n_{n+1}^{trial}$$

$$\alpha_{n+1}^p = \alpha_n^p + \sqrt{\frac{2}{3}} \cdot \gamma_{n+1}$$

$$\beta_1 = 1 - \frac{\phi_{n+1}^{trial}}{\|\xi_{n+1}^{trial}\|} \cdot \frac{1}{1 + \frac{h}{3\mu}}, \quad \beta_2 = \left[1 - \frac{\phi_{n+1}^{trial}}{\|\xi_{n+1}^{trial}\|} \right] \cdot \frac{1}{1 + \frac{h}{3\mu}}$$

$$C_{n+1}^{dev} = 2\mu\beta_1 \mathbb{P}_{sym} - 2\mu\beta_2 n_{n+1}^{trial} \otimes n_{n+1}^{trial}$$

Add spherical contributions to stress and moduli.

$$(\sigma_{n+1}) = dev(\sigma_{n+1}) + k tr(\epsilon_{n+1}^{total}) \cdot I$$

$$C_{n+1} = C_{n+1}^{dev} + k I \otimes I$$

Gauss quadrature

Integration formula for an element,

$$\int f(x,y)d\Omega = \int_{-1}^1 \int_{-1}^1 f(x(\xi,\eta),y(\xi,\eta))J(\xi,\eta) \cdot d\xi d\eta$$

For numerical integration, gauss quadrature method is used.

$$\int f(x,y)d\Omega = \sum_{i=1}^M f(x(\xi_i,\eta_i),y(\xi_i,\eta_i))J(\xi_i,\eta_i) * Weight$$

Where f is a function, and J is the jacobian determinant. Two sampling points in each unit coordinate direction hence locations and weights of gauss points are,

α	1	2	3	4
$\xi(\alpha)$	$-\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$-\frac{1}{\sqrt{3}}$
$\eta(\alpha)$	$-\frac{1}{\sqrt{3}}$	$-\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$	$\frac{1}{\sqrt{3}}$
Weight	1	1	1	1

Table 2: Locations and weight of gauss points

Applying the gauss quadrature method of numerical integration to find k^{ele} and $F_{internal}^{ele}$

$$k^{ele} = weight * B^T * C_t * B * 2 * \pi * (N_1 r_1 + N_2 r_2 + N_3 r_3 + N_4 r_4) * || Jacobian ||$$

$$F_{internal}^{ele} = weight * B^T * \sigma * 2 * \pi * (N_1 r_1 + N_2 r_2 + N_3 r_3 + N_4 r_4) * || Jacobian ||$$

Connectivity matrix is multiplied with elemental stiffness and internal forces to find the global stiffness matrix and internal forces. Since it's a displacement driven problem, $F^{external} = 0$

After finding global stiffness matrix and internal forces , we apply the essential boundary conditions.

1. $U_z = 0$ at $z = 0$
2. $U_r = 0$ at $r = 0$
3. $U_z = Compression\ load$ at $z = maximum\ height$

Reduced global stiffness matrix and internal forces,

$$G = -F_{internal}^{global}, \quad F_{external}^{global} = 0$$

Now we solve for displacement,

$$\Delta U = K^{-1}(G)$$

$$U_{k+1}^m = U_k^m + \Delta U$$

Where k is Newton Raphson iteration and m is timestep

This process continues until it is Converged with Newton Raphson method.

$$||G|| < 0.005 ||F_{internal}^{global}||, \quad ||\Delta U|| < 0.005 ||U||$$

4. PROGRAM STRUCTURE

4.1. PARAMETER IDENTIFICATION FLOWCHART

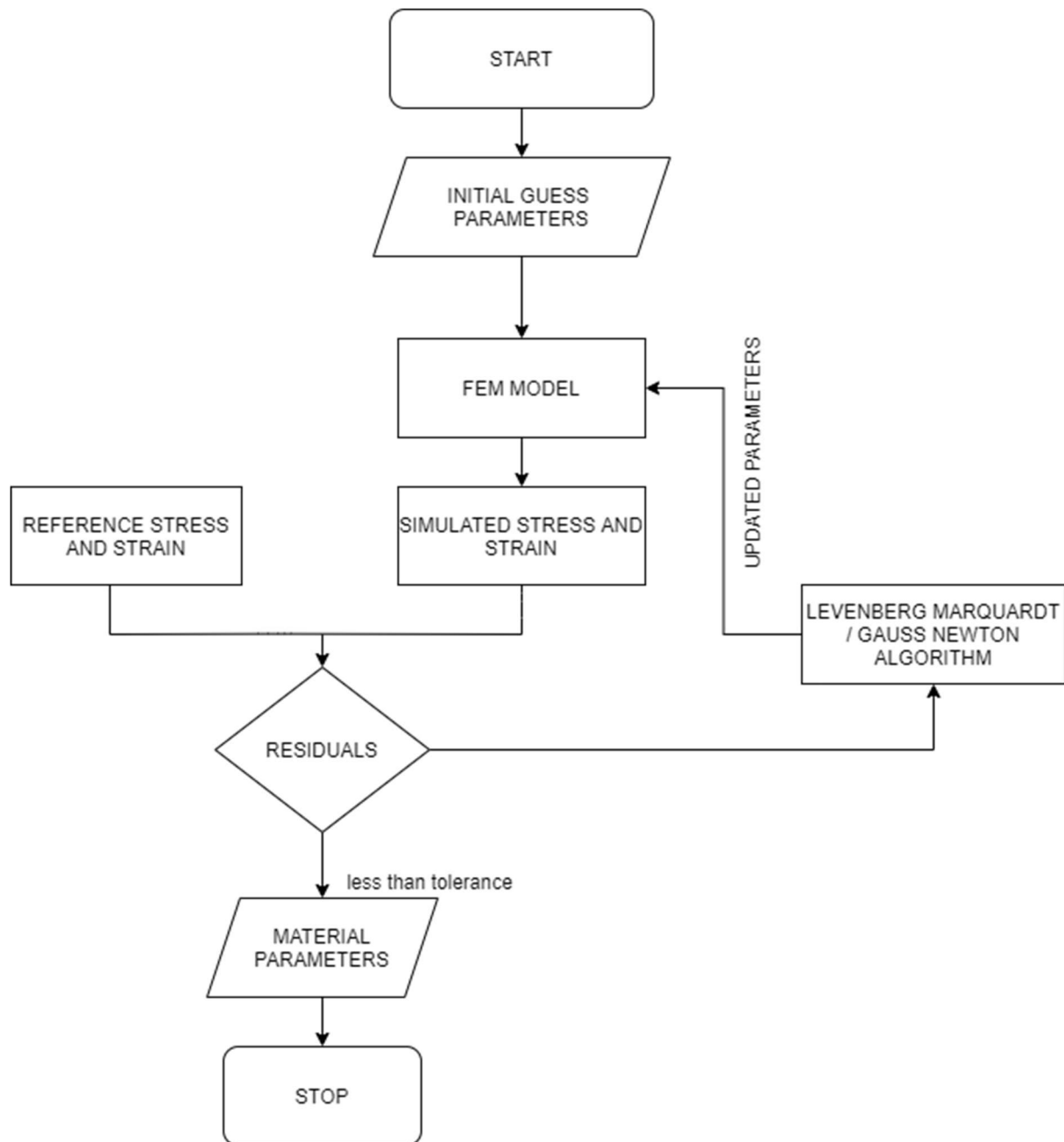


Figure 6 : Parameter Identification flowchart

4.2. FEM FLOWCHART

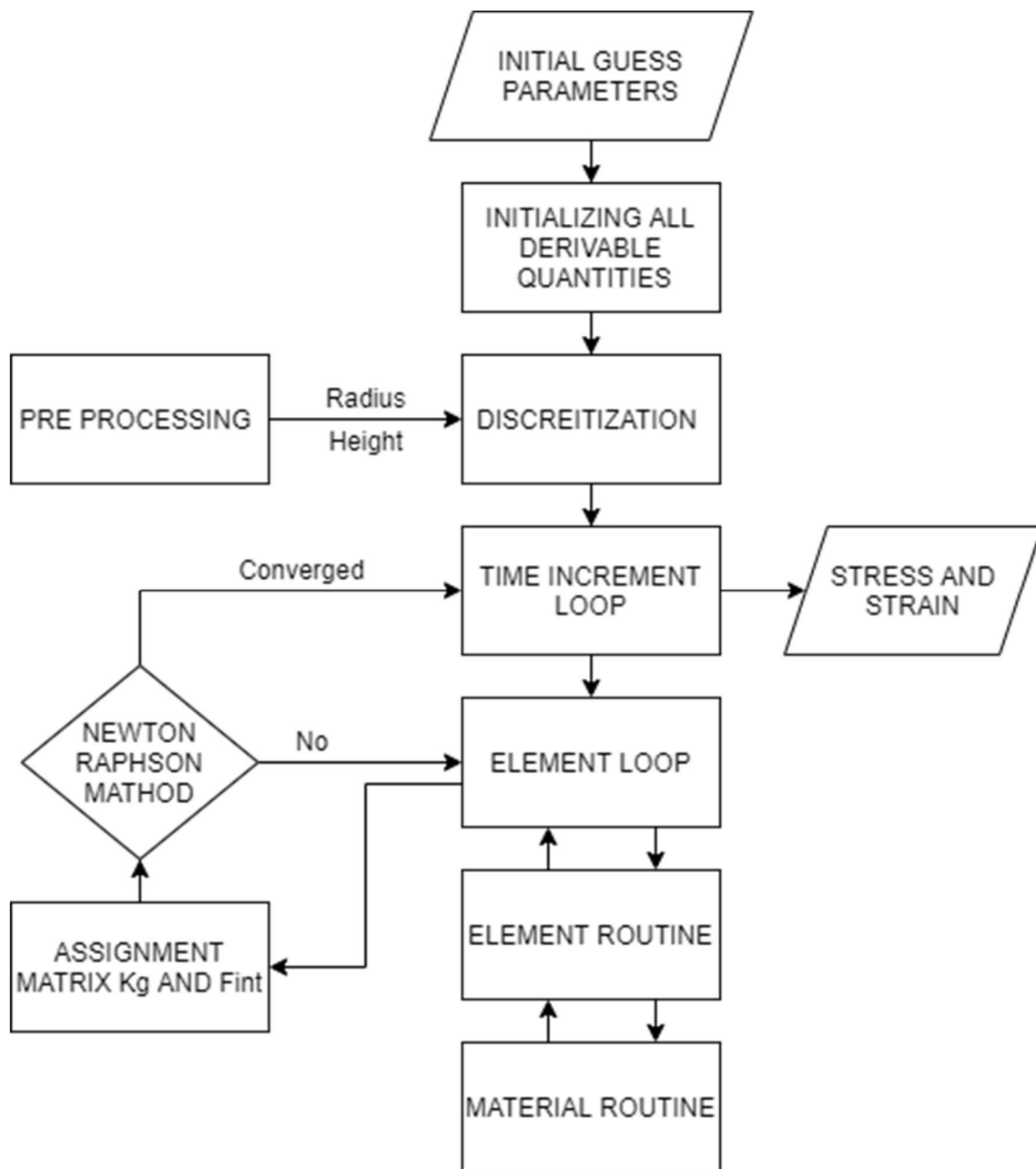


Figure 7: Finite element program flow chart

5. RESULTS OF TESTS AND DISCUSSION

TEST CASE : 1

Manual calculation test

AIM : To check the FEM implementations (elasticity) for 1 element- 2timesteps with manual calculation.

Manual calculations from the Chapter 3. (Finite Element Formulations) implementations are done manually.

Input parameters (Grade B500 steel [4]) :

- Young's modulus = 210000 Mpa
- Yield stress = 500 Mpa
- Hardening modulus = 250 Mpa
- Nonlinear hardening modulus = 250 Mpa
- Nonlinear Hardening exponent = 15
- External displacement = 0.003 mm (displacement driven problem)
- Timesteps : 2

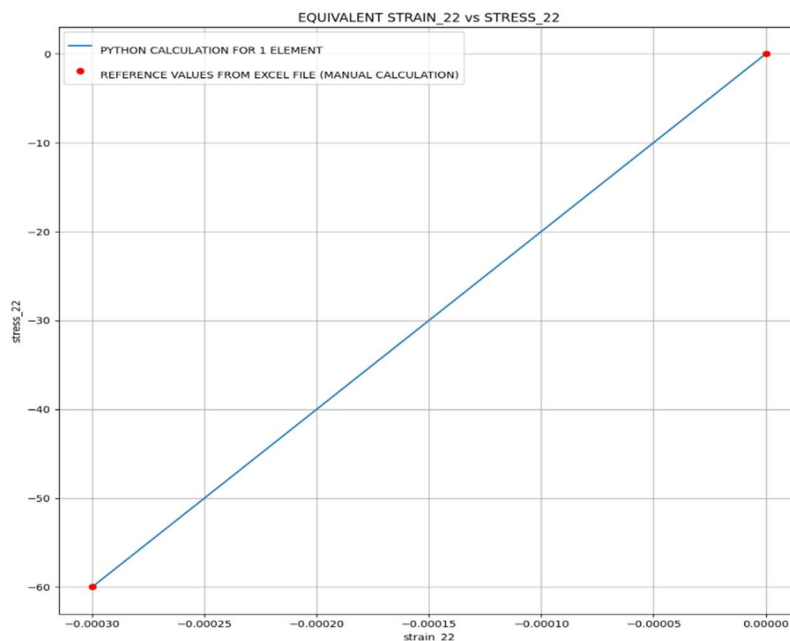


Figure 8: Strain vs Stress plot - Comparison of manual calculation and python implementations

DISCUSSION : Equivalent stress and strain from FEM implementations are compared with the manual calculation and results are verified.

TEST CASE : 2

Unit test

AIM : To check the individual blocks/functions of codes.

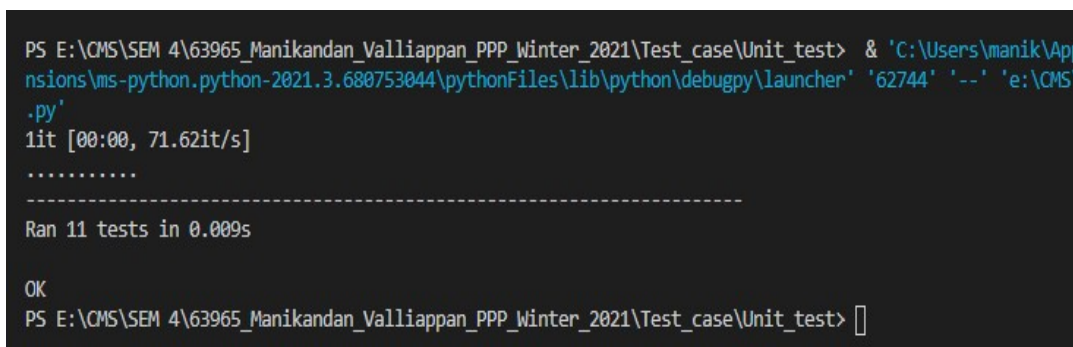
Unit testing is the testing method where individual blocks/functions of codes are tested.

Purpose is to validate each unit of code performs as expected.

Inputs : All input parameters are taken from the previous test case - Manual calculation test.

Functions tested are,

1. C_matrix for elastic case
2. Stress at elastic case
3. Pre-processing - radius and height values of solid cylinder
4. Assembly matrix
5. Shape functions
6. Differentiated shape functions
7. Nonlinear hardening function



```
PS E:\CMS\SEM 4\63965_Manikandan_Valliappan_PPP_Winter_2021\Test_case\Unit_test> & 'C:\Users\manik\AppData\Local\Microsoft\WindowsApps\python-2021.3.680753044\pythonFiles\lib\python\debugpy\launcher' '62744' '--' 'e:\CMS\SEM 4\63965_Manikandan_Valliappan_PPP_Winter_2021\Test_case\Unit_test\unit_test.py'
1it [00:00, 71.62it/s]
.....
-----
Ran 11 tests in 0.009s

OK
PS E:\CMS\SEM 4\63965_Manikandan_Valliappan_PPP_Winter_2021\Test_case\Unit_test> █
```

Figure 9 : Unit test verification

DISCUSSION : Each block of code tested and compared with known value from manual calculation test and results are verified.

TEST CASE : 3

Rigid body translation test

AIM : To check the implemented FEM formulations (Translation of system) for external displacement (displacement driven problem) with no boundary constraints.

Input parameters (Grade B500 steel [4]) :

- Young's modulus = 210000 Mpa
- Yield stress = 500 Mpa
- Hardening modulus = 250 Mpa
- Nonlinear hardening modulus = 250 Mpa
- Nonlinear Hardening exponent = 15
- External displacement = 15 mm (displacement driven problem)

RESULTS :

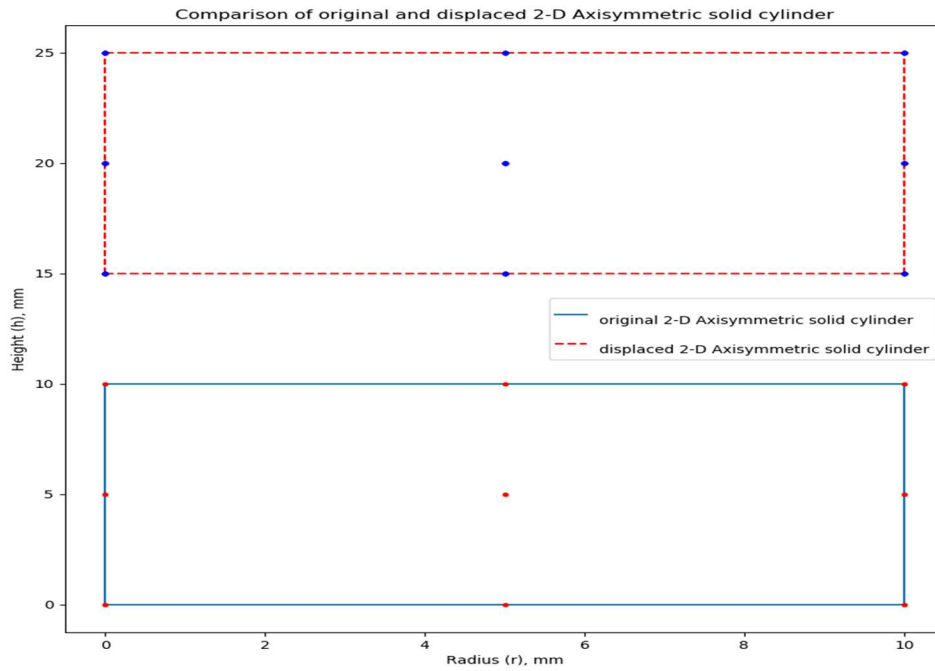


Figure 10: Rigid body translation test - comparison of original vs displaced 2-D axisymmetric solid cylinder.

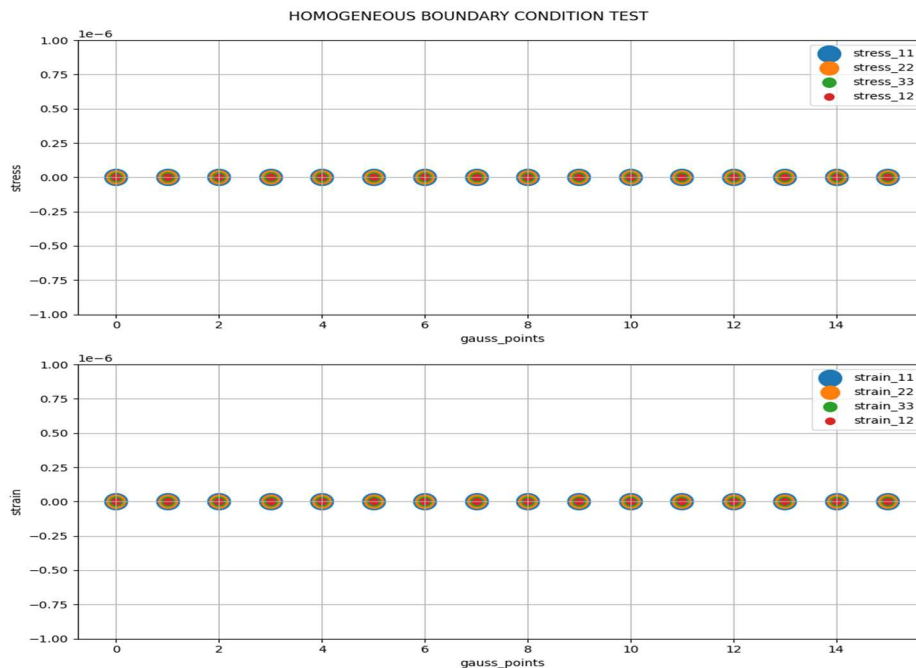


Figure 11: strain vs stress plot for homogeneous boundary condition test

DISCUSSION :

1. Zero stress and strain (figure 11) observed at all gauss points and system moved accordingly to the given external displacement.
2. Each node displaced 15 mm (figure 10) from its original position.

TEST CASE : 4

Homogeneous boundary condition test

AIM : To check the implemented FEM formulations for Homogeneous boundary constrains.

Homogeneous boundary condition test is the testing case where every part of the material deforms as the whole system does.

Input parameters (Grade B500 steel [4]) :

- Radius = 10 mm
- Height = 10 mm
- Young's modulus = 210000 Mpa
- Yield stress = 500 Mpa
- Hardening modulus = 250 Mpa
- Nonlinear hardening modulus = 250 Mpa
- Nonlinear Hardening exponent = 100
- External displacement = -0.5 mm (displacement driven problem and negative sign indicates compression)

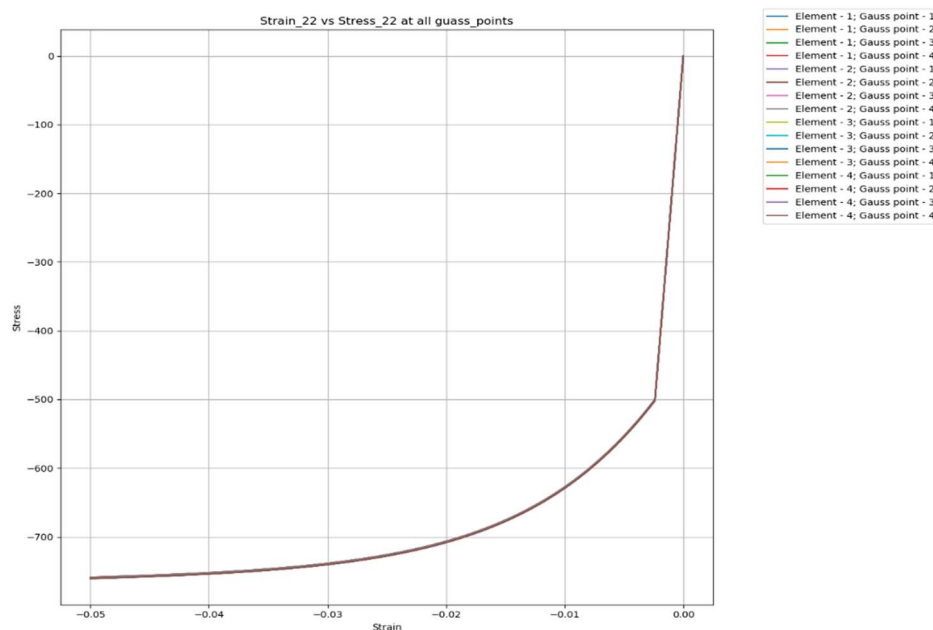


Figure 12: Strain Vs Stress plot of loading direction at all gauss points.

Results:

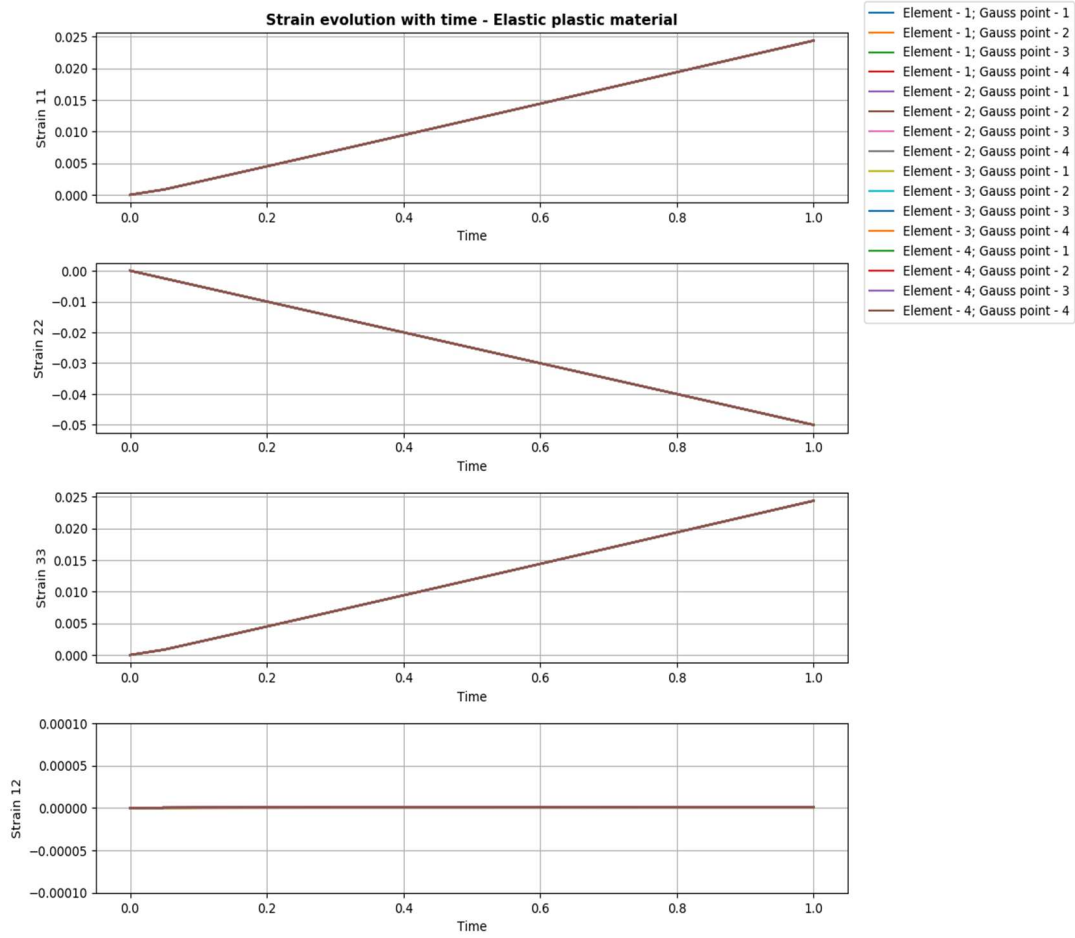


Figure 13: Strain evolution with time at all gauss points

DISCUSSION :

1. Strain_22 and Stress_22 → displacement loading direction.
2. (Strain_11, Stress_11), (Strain_33, Stress_33) → Poisson's effect
3. Strain_12 → nearly close to zero
4. At all gauss points, same stress and strain values are observed.

TEST CASE : 5

Voce hardening test.

AIM : To check the plastic behaviour with isotropic hardening with Voce hardening law

Inelastic isotropic nonlinear hardening can be described by the voce hardening law.

$$\sigma_y(\epsilon^{pl}) = \sigma_0 + \sigma_1 \epsilon^{pl} + \sigma_2 [1 - \exp(-n \epsilon^{pl})] \quad (5.1)$$

Where σ_0 is Yield stress, σ_1 is Linear hardening coefficient, σ_2 is Nonlinear hardening coefficient, n is Nonlinear hardening exponent and ϵ^{pl} is plastic strain.

Input parameters (Grade B500 steel [4]) :

- Young's modulus = 210000 Mpa
- Yield stress = 500 Mpa
- Hardening modulus = 250 Mpa
- Nonlinear hardening modulus = 250 Mpa
- Nonlinear Hardening exponent = 100
- External displacement = -0.5 mm (displacement driven problem and negative sign indicates compression)

RESULTS :

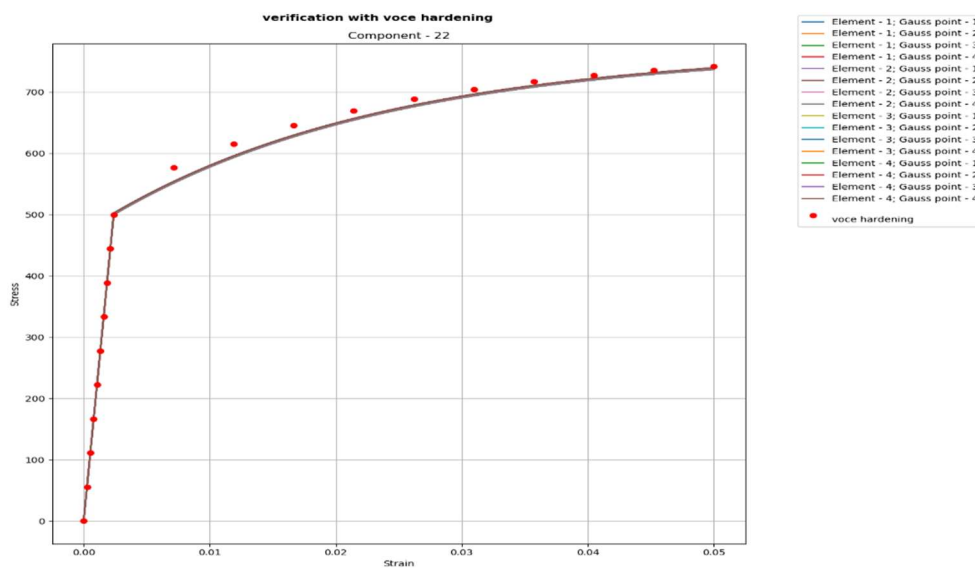


Figure 14: Verification of implemented FEM model with voce hardening law

DISCUSSION :

From the graph (graph multiplied with -1 and plotted), we can see that FEM simulated plastic behaviour with isotropic hardening is almost similar to the Voce hardening law. Hence the FEM implementation is verified.

TEST CASE : 6

Algorithmic test – Gauss Newton algorithm

AIM : To check the implementation of Gauss Newton algorithm works correctly for polynomial function.

In this test case, a polynomial function is taken into consideration to find the optimal Coefficients of the function. And the results are checked with the true polynomial function.

Polynomial function : $y = 10x^4 + 5x^3 + 5x^2 + 2x$

Objective function : $y = ax^4 + bx^3 + cx^2 + dx$

RESULTS :

Coefficients of function	a	b	c	d
Exact output coefficients	10	5	5	2
Input Coefficients(guess value)	50	60	70	80
Output from Gauss Newton algorithm implementation	9.65481596	5.34737363	5.11840999	1.8939671
Error in percentage	-3.4518%	6.9474726%	2.3681998%	5.301645%

Table 3 : Comparison of Exact and optimal coefficients of Gauss Newton method

PLOT :

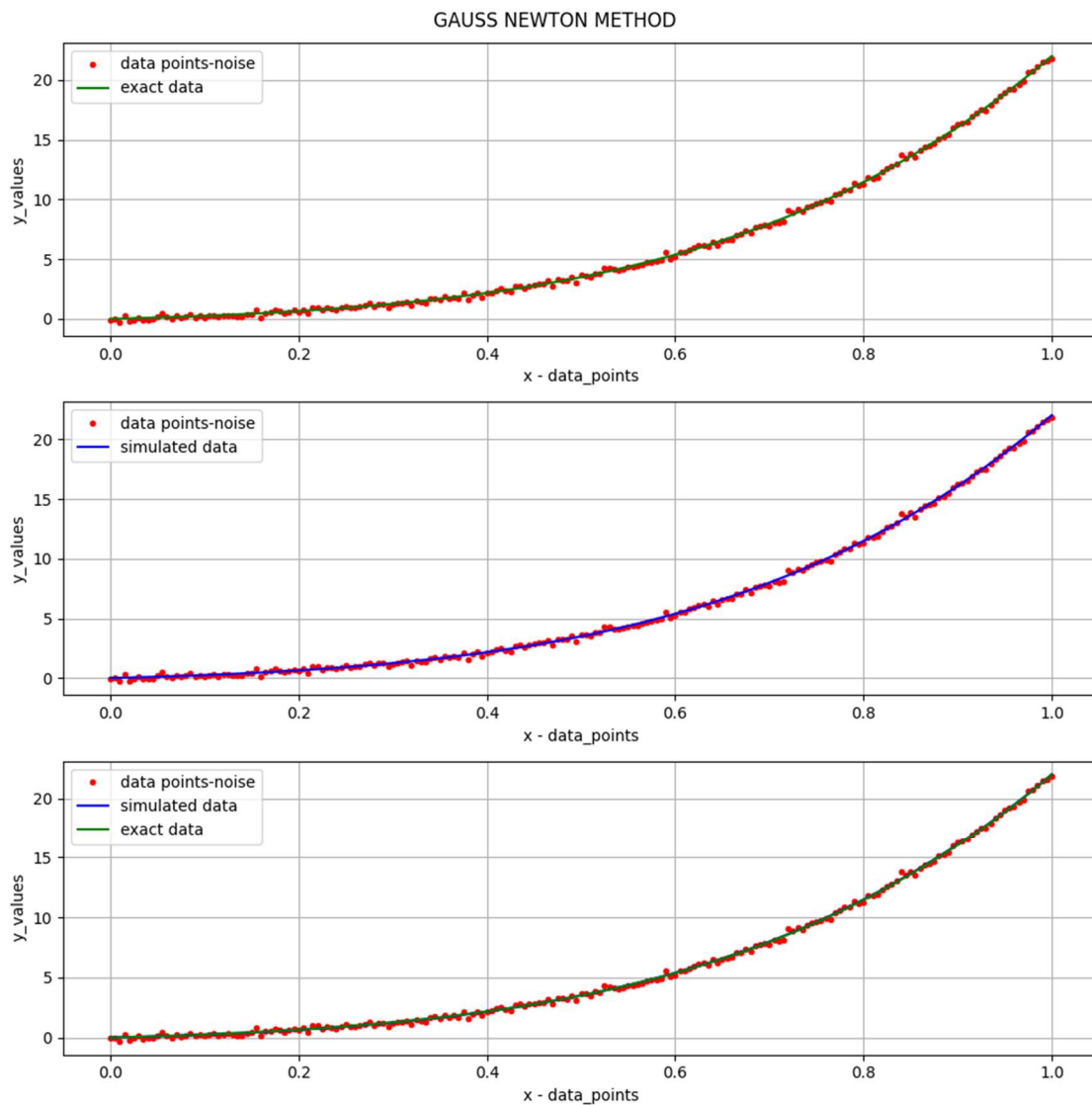


Figure 15: Gauss Newton method for polynomial function

Plot 1 : Data points (x) vs function (y) with exact coefficients

Plot 2 : Data points (x) vs function (y) with optimal coefficients obtained from the Gauss Newton algorithm

Plot 3 : Fit between exact and optimal coefficients of polynomial function.

DISCUSSION :

Output Coefficients from implemented Gauss Newton are the “optimum estimates” of the polynomial function which are nearly close to the true co-efficient.

TEST CASE : 7

Algorithmic test – Levenberg Marquardt algorithm

AIM : To check the implementation of Levenberg Marquardt algorithm works correctly for polynomial function.

In this test case, same polynomial function is taken into consideration to find the optimal Coefficients of the function. And the results are checked with the true polynomial function.

Polynomial function : $y = 10x^4 + 5x^3 + 5x^2 + 2x$

Objective function : $y = ax^4 + bx^3 + cx^2 + dx$

Results :

Coefficients of function	a	b	c	d
Exact output co-efficient	10	5	5	2
Input Coefficients(guess value)	450	800	20	45
Output from Levenberg Marquardt algorithm implementation	9.65481504	5.34737539	5.11840896	1.89396728
Error in percentage	3.4518496%	6.4961848%	2.313393%	5.5984451%

Table 4: Comparison of Exact and optimal coefficients of Levenberg Marquardt method

The following graph explains the implemented Levenberg Marquardt algorithm which gives optimal Coefficients of the function.

Plot 1 : Data points (x) vs function (y) with exact coefficients

Plot 2 : Data points (x) vs function (y) with optimal coefficients obtained from the Levenberg Marquardt algorithm

Plot 3 : Curve fit between exact and optimal coefficients of polynomial function.

Plot 4 : Results are compared with SciPy fit.

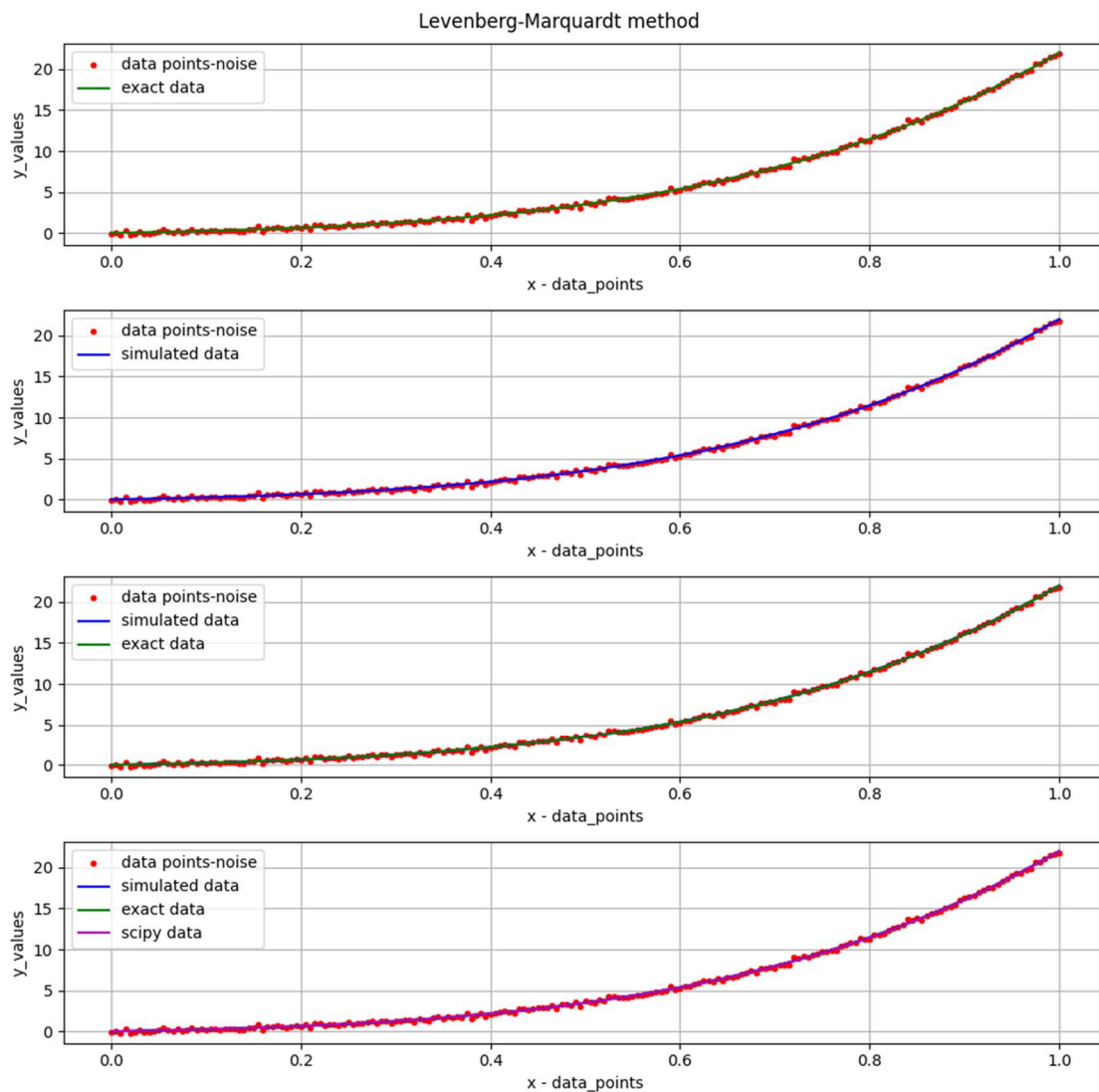


Figure 16: Gauss newton method for polynomial function

DISCUSSION :

Output coefficients from implemented Levenberg Marquardt are the “optimum estimates” of the polynomial function which are nearly close to the true co-efficient.

TEST CASE 8 :

Parameter Identification of Elastic-Plastic material and its FEM implementations (Levenberg Marquardt algorithm)

AIM : To find Optimal parameters of Elastic plastic material using Levenberg Marquardt algorithm

Material Parameters optimized in this test cases are,

1. Young's Modulus
2. Yield stress
3. Hardening modulus
4. Nonlinear hardening modulus
5. Nonlinear Hardening exponent

Initial guess of parameters is given as an input to run the FEM elastic plastic model and Levenberg Marquardt algorithm will optimize the parameters (Grade B500 steel [4]) according to the reference data. Here algorithm fits the curve according to the reference data points and finds the optimal material parameters.

Material parameters	Exact parameters	Initial Guess Parameters	Obtained Optimal parameters
Young's modulus (E)	210000 (Mpa)	5000 (MPa)	2100692.24(MPa)
Yield stress (σ_y)	500 (MPa)	50 (MPa)	499.935438 (MPa)
Linear hardening modulus (h)	250 (MPa)	50 (MPa)	249.900651 (MPa)
Nonlinear hardening modulus (σ_2)	250 (MPa)	50 (MPa)	250.075907 (MPa)
Hardening exponent (η)	15	5	14.9922716

Table 5: Comparison of exact and optimal parameters obtained from Levenberg Marquardt algorithm

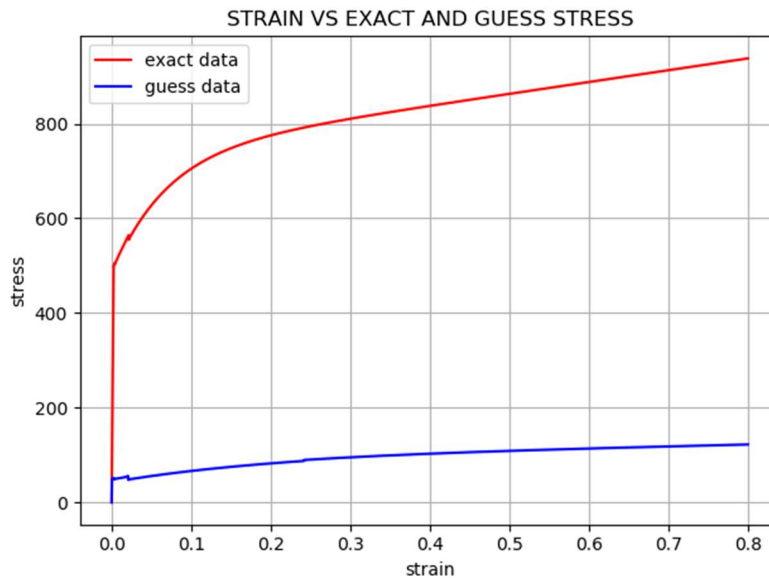


Figure 17: Comparison of Strain vs Stress for exact and guess values

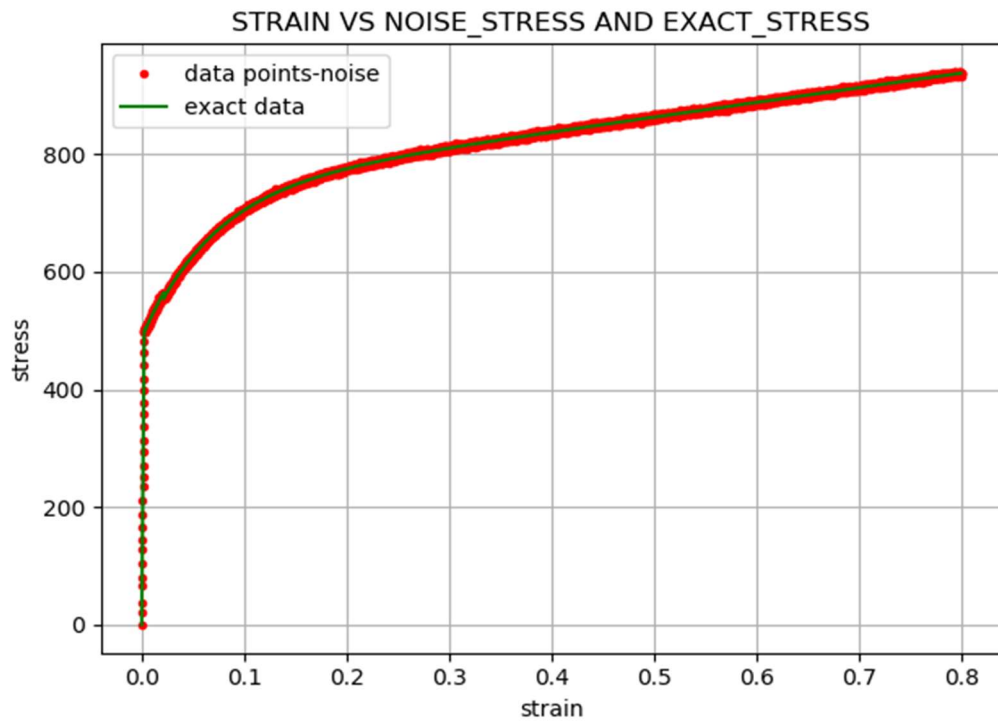


Figure 18: Comparison of Strain vs Stress for noise and exact values

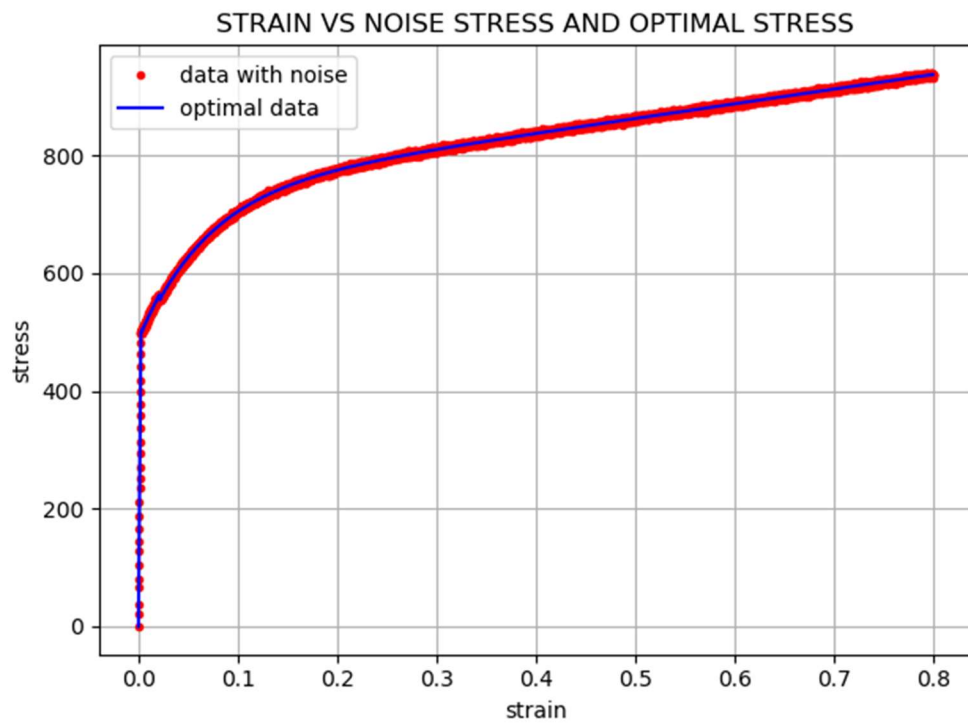


Figure 19 : Comparison of Strain vs Stress for noise and optimal values

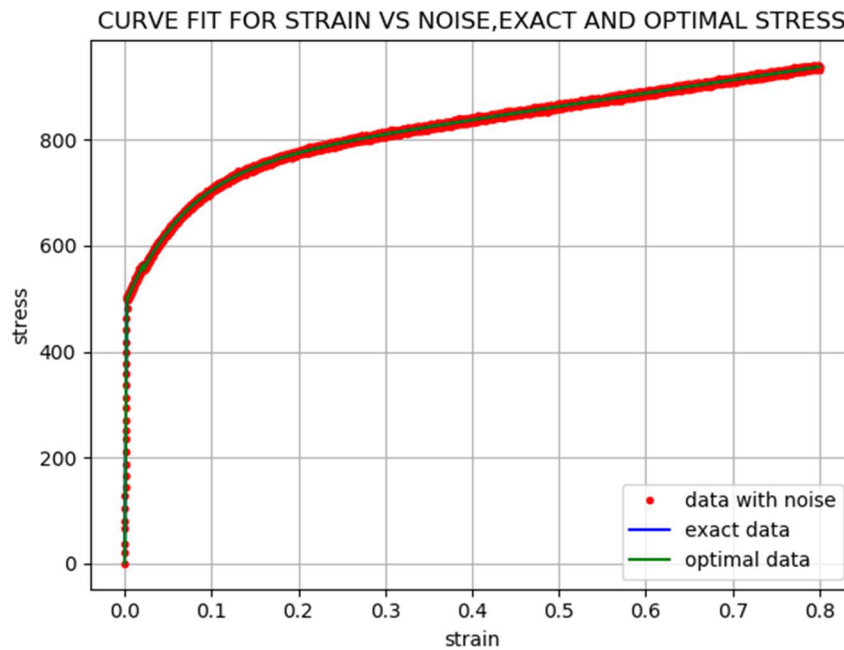


Figure 20: Curve fit for Strain vs Stress for noise, exact and optimal values.

Plot 1 : Strain vs Stress with initial guess parameters

Plot 2 : Strain vs Stress with exact optimal parameters

Plot 3 : Strain vs Stress with optimal parameters obtained from the Levenberg Marquardt algorithm

Plot 4 : Fit between exact and optimal parameters of function.

DISCUSSION:

Output Parameters from implemented Levenberg Marquardt are the “optimum parameters” of the Elastic-Plastic material (Grade B500 steel [4]) which are close to the true parameters.

TEST CASE 9 :

Parameter Identification of Elastic-Plastic material and its FEM implementations (Gauss Newton algorithm)

AIM : To find Optimal parameters of Elastic plastic material using Gauss Newton algorithm

Material Parameters optimized in this test cases are,

1. Young's Modulus
2. Yield stress
3. Hardening modulus

4. Nonlinear hardening modulus
5. Nonlinear hardening exponent

Initial guess of parameters is given as an input to run the FEM elastic plastic model and Gauss Newton algorithm will optimize the parameters (Grade B500 steel [4]) according to the given reference data.

Here algorithm fits the curve according to the reference data points and finds the optimal material parameters.

RESULTS

Material parameters	Exact parameters	Initial Guess Parameters	Obtained Optimal parameters
Young's modulus (E)	210000 (Mpa)	150000 (MPa)	210000.000(Mpa)
Yield stress (σ_y)	500 (MPa)	300 (MPa)	499.950966 (MPa)
Linear hardening modulus (h)	250 (MPa)	300 (MPa)	249.895552 (MPa)
Nonlinear hardening modulus (σ_2)	250 (MPa)	300 (MPa)	250.0060161(MPa)
Hardening exponent (η)	15	20	14.9909995

Table 6: Comparison of exact and optimal parameters obtained from Gauss Newton algorithm

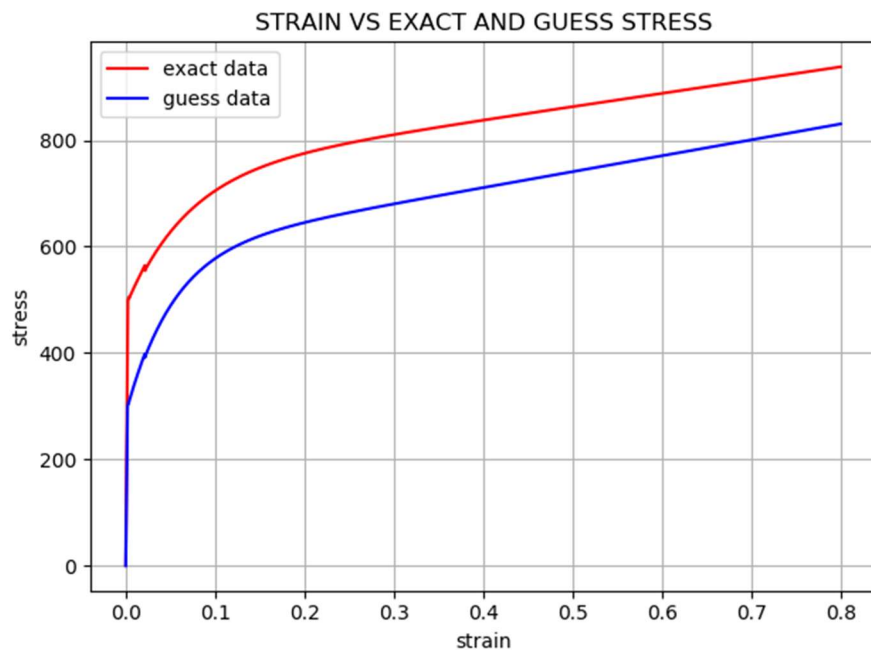


Figure 21: Comparison of Strain vs Stress for exact and guess values

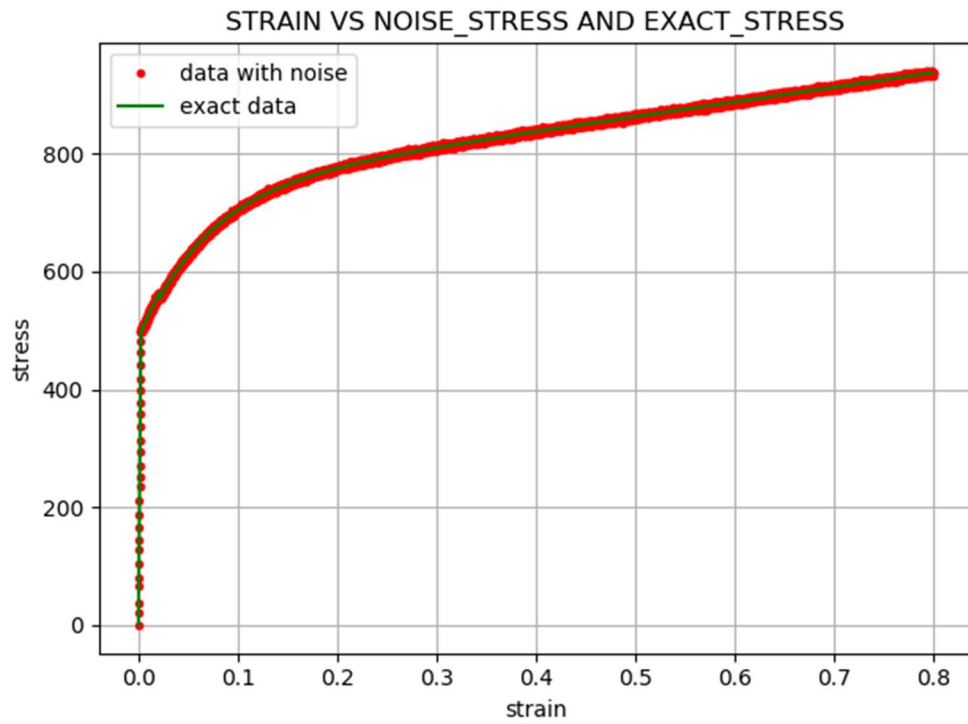


Figure 22: Comparison of Strain vs Stress for noise and exact values

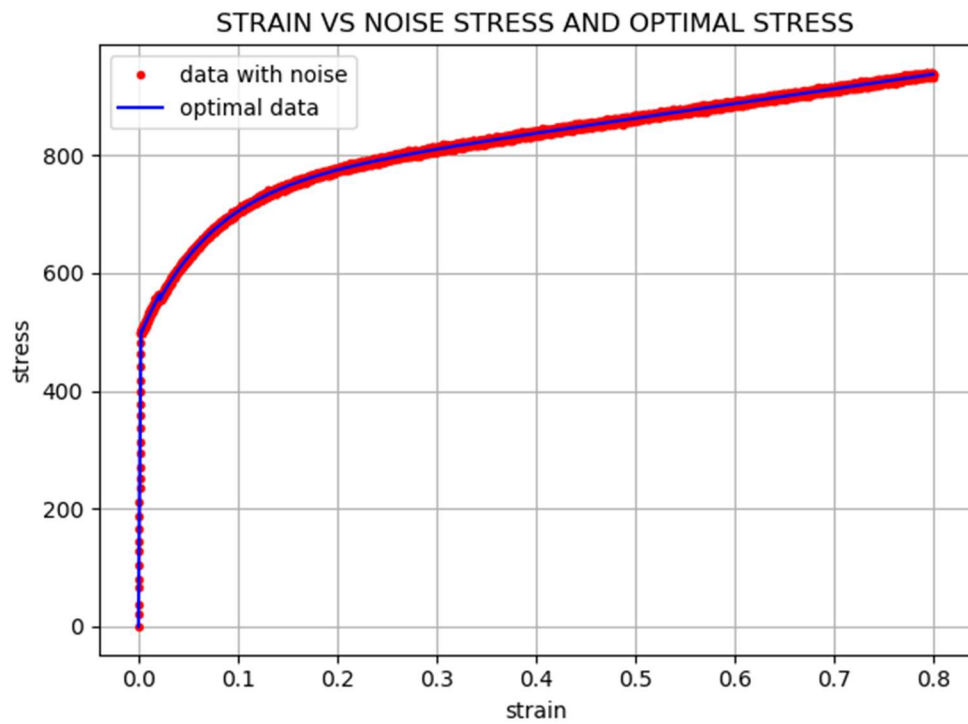


Figure 23: Comparison of Strain vs Stress for noise and optimal values

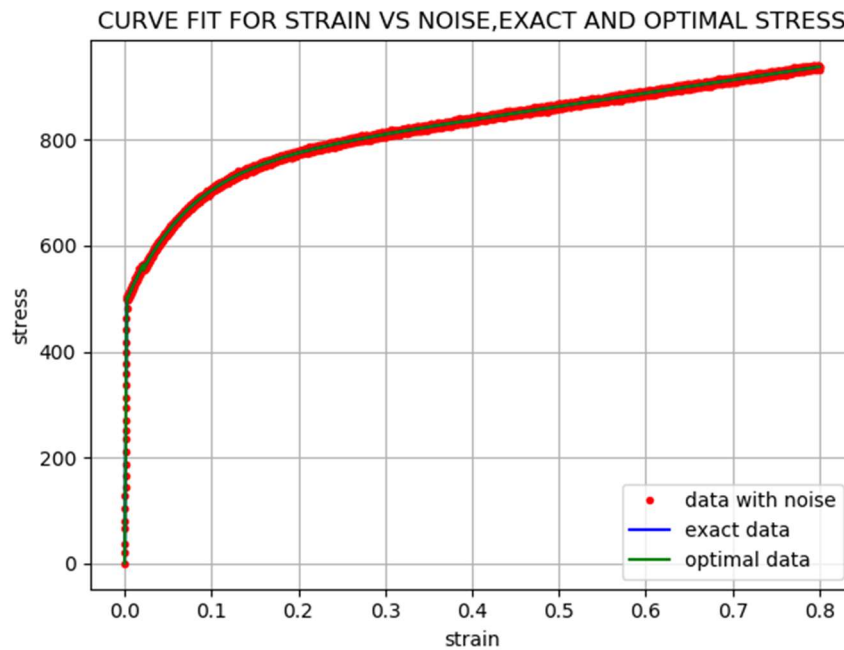


Figure 24: Curve fit for Strain vs Stress for noise, exact and optimal values

Plot 1 : Strain vs Stress with initial guess parameters

Plot 2 : Strain vs Stress with exact optimal parameters

Plot 3 : Strain vs Stress with optimal parameters obtained from the Gauss Newton algorithm

Plot 4 : Curve fit between exact and optimal parameters of function.

DISCUSSION:

Output Parameters from implemented Gauss Newton algorithm are the “optimum parameters” of the Elastic-Plastic material (Grade B500 steel [4]) which are close to the true parameters.

TEST CASE 10 :

SciPy curve fit test

SciPy is an open-source python package with library of functions to fit models to data.

AIM : To check the implemented algorithms with SciPy

Material Parameters optimized in this test cases are,

1. Young's modulus
2. Yield stress
3. Hardening modulus
4. Nonlinear hardening modulus

5. Nonlinear hardening exponent

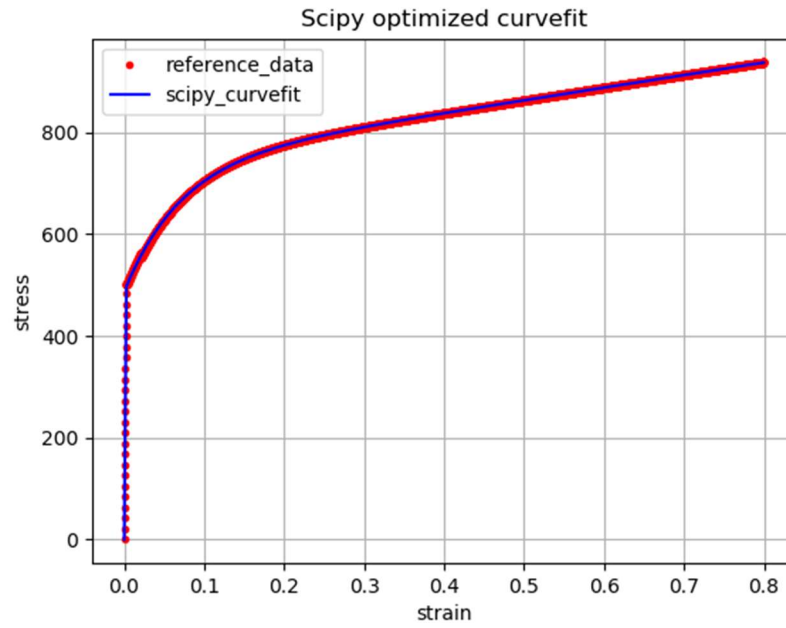


Figure 25: SciPy optimized curve fit compared with reference data

Optimized parameters obtained from SciPy test are compared with implemented algorithm below:

Material parameters	Exact parameters	Levenberg Marquardt optimized Parameters	Gauss Newton optimized Parameters	SciPy Optimal parameters
Young's modulus (E)	210000 (Mpa)	2100692.24(MPa)	210000.000(Mpa)	198745.059 (Mpa)
Yield stress (σ_y)	500 (MPa)	499.935438 (MPa)	499.950966 (MPa)	487.42614538 (MPa)
Linear hardening modulus (h)	250 (MPa)	249.900651 (MPa)	249.895552 (MPa)	247.23538896 (MPa)
Nonlinear hardening modulus (σ_2)	250 (MPa)	250.075907 (MPa)	250.0060161(MPa)	252.38504976 (MPa)
Hardening exponent (η)	15	14.9922716	14.9909995	14.32624873

Table 7: Comparison of exact and optimal parameters obtained from Levenberg Marquardt. Gauss Newton algorithm and scipy

DISCUSSION:

Output Parameters from SciPy is compared with implemented Levenberg Marquardt and Gauss Newton algorithms of the elastic plastic material which are close to the true parameters. Hence implemented two algorithms are verified.

INFERENCE:

From the above results, we can understand that both the Gauss Newton and Levenberg Marquardt algorithm solve nonlinear least-square problems by reducing the sum of squared function values. However, the Levenberg Marquardt algorithm acts like the Gauss Newton algorithm when the initial guess is close to the exact solution and acts like the gradient descent method when far from the solution. Hence it takes more time to compute the exact parameters when far from the true solution, but it can work for any guesses even far from the exact solution. Levenberg Marquardt algorithm can find the solution even after it starts very far from the final minimum, Whereas Gauss Newton finds the solution when the initial guess closer to the exact solution.

6. TASKS PROPOSED Vs TASKS ACHIEVED

Proposed : Parameter Identification of Elastic-Viscoplastic material and its FEM implementations

Achieved : Parameter Identification of Elastic-Plastic material and its FEM implementations

Reasons for change of material model:

1. Visco-plastic part remains un-converged and the evolution of strain along the loading direction (ϵ_{22}) was positive as opposed to the negative compression.
2. The same behaviour was observed in the stress-strain plot where stress values during the visco-plastic regime for some gauss points were positive (Should be negative for a pure compression loading)
3. The delta sigma (σ_{22}) (in the visco-plastic part) calculated was approximately 2 times higher than the sigma (σ_{22}) calculated in the elastic regime. This poses a sign for the possibility of an error in my material routine which was not able to figure it out.
4. Faced few difficulties in updating the displacements when the material is in Viscoplastic regime due to intrinsic time parameters involved in it.
5. Since the main objective is to identify material parameters by using Levenberg Marquardt and Gauss Newton algorithm and due to above mentioned obstacles, material model is switched from Elastic-Viscoplastic to Elastic-Plastic material

Results obtained from FEM implementations of Elastic-Viscoplastic material :

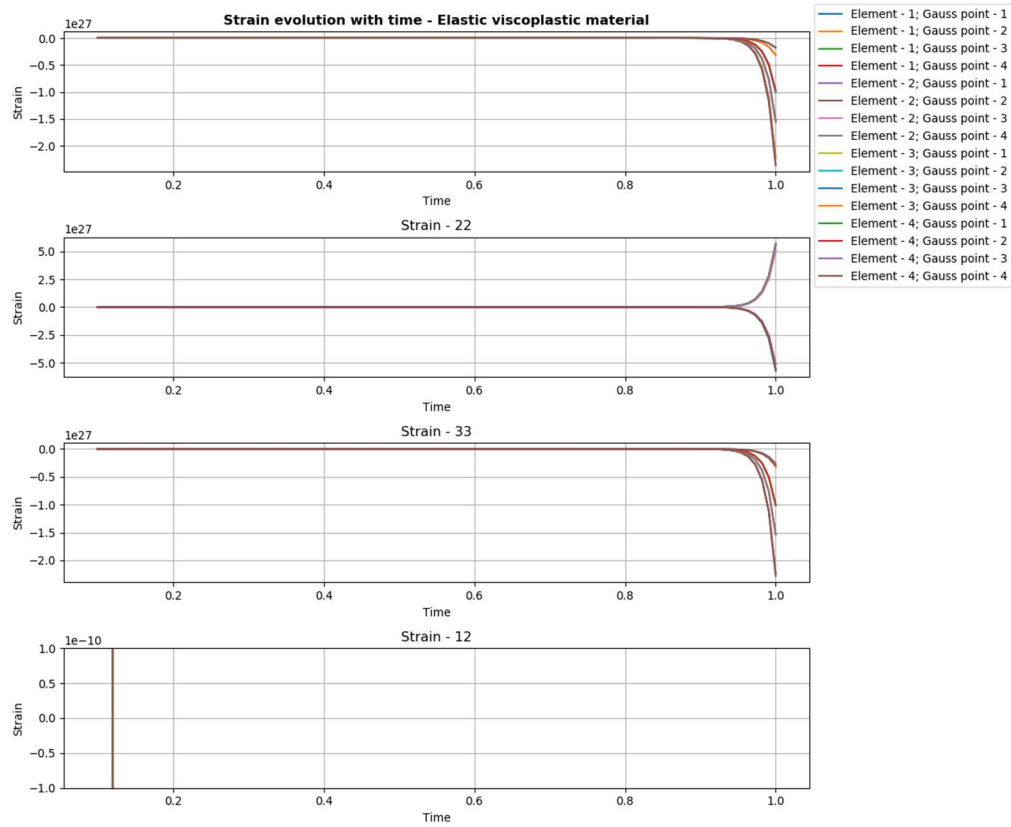


Figure 26 : Strain evolution of Elastic-Viscoplastic material with respect to time

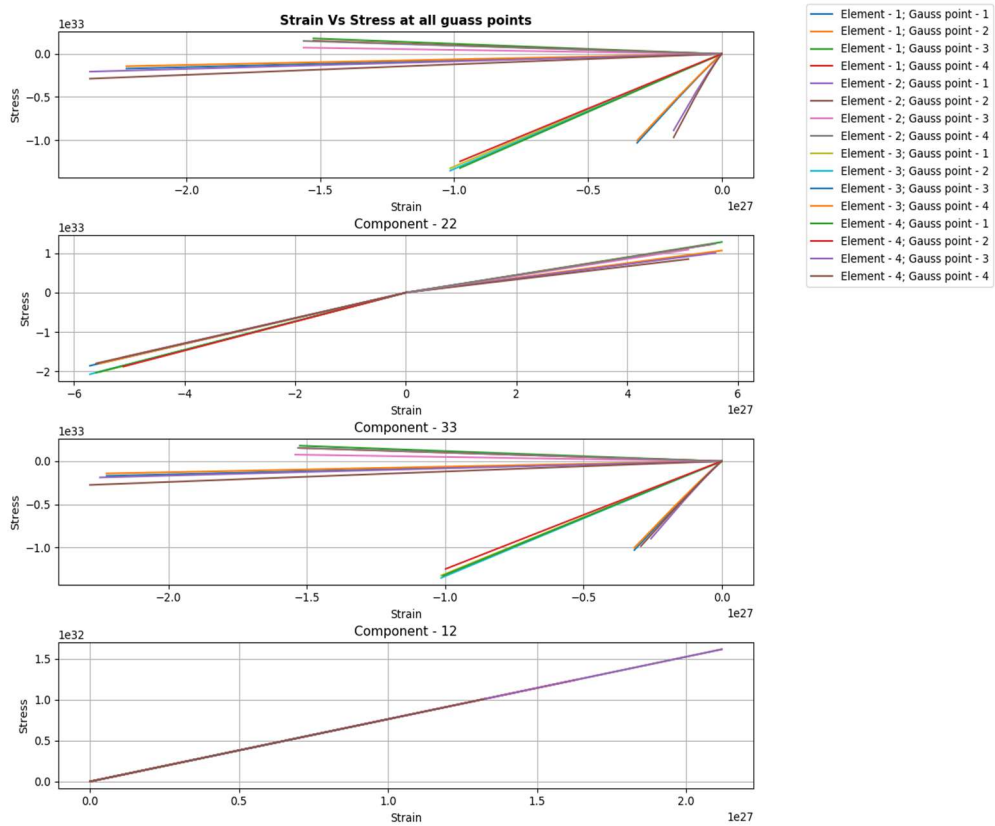


Figure 27 : Strain vs Stress plot at all gauss points

7. USER MANUAL

REQUIREMENTS :

Programming language : Python 3.8.0

To download : <https://www.python.org/downloads/release/python-383/>

Libraries : Math, NumPy, Matplotlib, Tqdm and SciPy

Version : (1.17.4) and above

To download :

Math - <https://pypi.org/project/python-math/>

NumPy - <https://pypi.org/project/numpy/>

SciPy - <https://www.scipy.org/install.html>

Matplotlib - <https://pypi.org/project/matplotlib/>

Tqdm - <https://pypi.org/project/tqdm/>

STEP 1 : Open the terminal and change the working directory to where the *main_program.py* file located.

STEP 2 : Run the program by executing the following command.

```
python3 main_program.py
```

Note : All necessary files are included in the main folder itself. Therefore, ensure first *elastic_plastic.py*, *experimental_data.txt*, *stress_strain_noise.txt*, *levenberg_elastic_plastic.py*, *gauss_newton_elastic_plastic.py* are in the same location as the *main_program.py* file.

Two algorithms (Levenberg Marquardt and Gauss Newton algorithms for Elastic-Plastic material) are implemented in this work, therefore choose which algorithm to run.

STEP 3 : Enter 0 for Levenberg Marquardt algorithm and 1 for Gauss Newton algorithm

```
!!! WELCOME !!!
```

```
PARAMETER IDENTIFICATION OF ELASTIC -PLASTIC MATERIAL AND ITS FEM  
IMPLEMENTATIONS
```

```
Please select which algorithm to run for optimizing the material parameters :
```

```
a). Levenberg Marquardt algorithm, Enter 0
```

b). Gauss Newton algorithm, Enter 1

Please the enter the value 0 or 1 to optimize the material parameters :

Note: If entered number is not 0 or 1 then program will display warning message

!!! Warning !!!

Entered number is not 0 or 1

STEP 4 : After selecting the algorithm, program will ask for initial guess parameters.

Enter an initial guess for Young's modulus :

Enter an initial guess for Yield stress:

Enter an initial guess for Hardening modulus:

Enter an initial guess for Nonlinear Hardening modulus :

Enter an initial guess for Nonlinear Hardening exponent :

STEP 5 : Enter the initial guess values for all five parameters.

Note : Make sure that initial guess should not deviate much more from the reference data.

If initial guess value is too large than the original parameter values then following error message will display in the terminal.

!!! Warning !!!

Entered value is not in the trust region radius

Please enter the value close to the reference data

Best guess range would be

Material parameters	Levenberg Marquardt algorithm	Gauss Newton algorithm
Young's modulus (E)	100000 – 300000 (Mpa)	150000 – 220000 (Mpa)
Yield stress (σ_y)	10 – 700 (Mpa)	300 – 600 (Mpa)
Linear hardening modulus(h)	10 – 600 (Mpa)	300 – 600 (Mpa)
Nonlinear hardening modulus (σ_2)	10 – 600 (Mpa)	300 – 600 (Mpa)
Hardening exponent (η)	2 – 25	7 – 20

Table 8 : Range for guess parameters

STEP 6 : Upon successful execution of *main_program.py*, program will show optimized parameter values in the terminal and generate a *.txt* file and four *.png* files (for each algorithm based on selection) in the same folder. The *.txt* file contains optimized parameter values information and *.png* file shows the plot for exact and optimized parameters.

After execution, following files will be generated,

1. “<algorithm_name>_parameter_identification_of_elastic_plastic_material.txt” - contains detailed information of optimized material parameters.
2. “<algorithm_name>_strain_vs_exact_and_guess_stress.png” –shows comparison of Strain vs Stress for exact and guess values
3. “<algorithm_name>_strain_vs_noise_and_exact_stress.png” – shows comparison of Strain vs Stress for noise and exact values
4. “<algorithm_name>_strain_vs_noise_and_optimal_stress.png” - shows comparison of Strain vs Stress for noise and optimal values
5. “<algorithm_name>_curve_fit_for_strain_vs_noise_exact_and_optimal_stress.png” - shows curve fit for Strain vs Stress for noise, exact and optimal values.

Note : <algorithm_name> (based on the selection) will be added at each above-mentioned file names.

8. LITERATURE

- [1] Diehl, Ted. "Modeling of elastic-viscoplastic behavior and its finite element implementation." (1988).
- [2] Ghoneim, H., and Yu Chen. "A viscoelastic-viscoplastic constitutive equation and its finite element implementation." *Computers & Structures* 17.4 (1983): 499-509.
- [3] Zienkiewicz, Olgierd Cecil, et al. "The finite element method: solid mechanics. Vol. 2. Butterworth-heinemann, 2000."
- [4] Hingorani, R., et al. "Loss of ductility and strength of reinforcing steel due to pitting corrosion." *Proceedings of the 8th International Conference on Fracture Mechanics of Concrete and Concrete Structures FraMCoS-8*, Toledo, Spain. 2013.
- [5] Prof. Dr.-Ing. Björn Kiefer. Lecture "Plasticity." Technische Universität Bergakademie Freiberg.
- [6] Dr.-Ing. Martin Abendroth. Lecture "Parameter Identification in Nonlinear Solid Mechanics." Technische Universität Bergakademie Freiberg.

9. GIT LOG

Last and first commits are mentioned below,

- 1. commit 8efb94171b15607d3638af67564ede1e6d424284
Author: MANIKANDAN VALLIAPPAN <rammanish846@gmail.com>
Date: Wed Apr 14 23:40:48 2021 +0200
final check - manual calculations
- 2. commit c8401728b747bb596700440cddd61f88d137d117
Author: MANIKANDAN VALLIAPPAN <rammanish846@gmail.com>
Date: Wed Oct 28 11:51:18 2020 +0100
calculate_linear_regression_using_least_square_method

Total number of commits = 144

Note : External text file “*git_log.txt*” is attached with this report.